The Nice Data Systems SPA Framework

We develop web applications. To minimize back-and-forth communication with the server, we wanted to create single-page applications (SPAs). This approach allows us to work on only a portion of the displayed page.

There are many tools available to achieve this goal (such as HTMX or UnPoly). They deserve our full attention.

However, in our case, they require significant customization to bring our applications into compliance.

We thrive on challenges and are highly curious. That is why we are developing our own framework. The “Nice Data Systems SPA Framework” is therefore being developed according to our ideas and needs.

The products we develop operate in the following environment:

- server: Python (Flask, Django, FastAPI, ...)
- client: Standard browser (JavaScript, CSS, ...)

Our framework consists of two parts:

- Python Middleware (tailored for Flask): FlaskMiddleware
- a JavaScript library: ndspalib.js

The **middleware** extends Flask’s functionality, and the JavaScript **library** enables collaboration with the middleware, thereby implementing the SPA paradigm.

Warning : The current version (1.0.10-dev) is not yet ready for production, but allows us to attempt porting (often from HTMX and sometimes from UnPoly).

This development experience has allowed us to gain a deeper understanding of how JavaScript and Python application servers work. This has allowed us to expand our skills, which is always welcome.

As always, working alone can lead to a lack of direction. So we would love to hear your comments, suggestions, and feedback. Please don’t hesitate to share them!

> Translated from french with DeepL.com (free version)

Installation

...

Initialization

At startup, elements having the `nd-init="url"` attribute are updated with the server response (an HTML `String`). After this step, the `nd-init="url"` is removed useless reloads.

Forms

Possible action :

- Submit : the form is sent to the server via a POST request, then it is closed

- Apply : the form is sent to the server via a POST request, then it remains open
- Dismiss : the form is closed
- Clear: reset all fields
    - Revert: restore data from last state

To implement :

- dirty flag : set to true whenever a form field changes

```
<form nd-form
```

Dialogs

Dialogs are displayed as modal on top of the current window. The displayed elements are :

- A title
- A body
- An accept button
- A dismiss button

To supply all this information, a JSON data structure is used :

```
detail = {
    title: 'The title',      // The dialog's title
    body: 'The message',     // The dialog's body
    buttons: {
        accept: 'OK',           // The 'accept' button's label
        dismiss: 'Cancel'       // The 'dismiss' button's label
    }
}
```

Polling

Elements with an `[nd-poll]` attribute are reloaded from the server periodically.

Attributes

Attribute	Default	Meaning
`nd-poll`		Elements with an `[nd-poll]` attribute are reloaded from the server periodically.
`nd-url`		URL to poll
`nd-target`		Target selector (`#id`, `.class`). If not specified, the inner content is updated

Attribute	Default	Meaning
`nd-interval`	10000	Poll interval in milliseconds

Examples

Poll the `/poll` url every `3000` ms (3 seconds) and update all elements having the `target` class.

```
<div nd-poll nd-url="/poll" nd-target=".target" nd-interval="3000" class="mb-3 border p-1"></div>
```

Poll the `/poll` url every `5000` ms (5 seconds) and update the element having the `target-c` id.

```
<div nd-poll nd-url="/poll" nd-target="#target-c" nd-interval="5000" class="mb-3 border p-1"></div>
```

Poll the `/poll` url every `10000` ms (10 seconds which is the **default** value) and update the the inner content of the element itself since no `nd-target` is specified.

```
<div nd-poll nd-url="/poll" class="mb-3 border p-1"></div>
```

Links & buttons

```
<div class="mb-3">
    <button nd-link nd-url="/poll" nd-target=".target" class="btn btn-primary">A</button>
    <button nd-link class="btn btn-primary">B</button>
    <button nd-link nd-url="/test" nd-target="#target-c" class="btn btn-primary">C</button>
</div>
```

Select

Attributes

Attribute	Default	Meaning
`nd-switch`		`<select>` elements with an `nd-switch` will update targets on option change.
`nd-url`		URL to poll
`nd-target`		Target selector (`#id`, `.class`). If not specified, the inner content is updated
`nd-interval`	10000	Poll interval in milliseconds

Examples

```
<div class="row mb-3 align-items-center">
    <div class="col-2">
        <select class="form-select" aria-label="Default select example" nd-
switch=".numbers">
            <option>-- Select a value --</option>
            <option value="1">One</option>
            <option value="2">Two</option>
            <option value="3 with love">Three</option>
            <option value="4">Four</option>
        </select>
    </div>
    <div class="col-auto">
        <!-- Show if there is a value -->
        <div class="col-form-label numbers form-control" nd-show-for="*">Value :</div>
    </div>
    <div class="col-auto">
        <!-- Sync with source selector value -->
        <div class="numbers border-success form-control" nd-show-for="*" nd-sync></div>
    </div>
    <div class="col-auto">
        <!-- Sync with server url -->
        <div class="numbers border-success form-control" nd-show-for="*" nd-sync nd-
url="/dummy"></div>
    </div>
    <div class="col-auto">
        <!-- Hide if there is a value -->
        <div class="numbers border-danger form-control" nd-hide-for="*">No value !</div>
    </div>
</div>
```